On Reporting Performance and Accuracy Bugs for Deep Learning Frameworks: An Exploratory Study from GitHub

Guoming Long
Department of Computer Science
Loughborough University, UK
g.long@lboro.ac.uk

Tao Chen*
Department of Computer Science
Loughborough University, UK
t.t.chen@lboro.ac.uk

ABSTRACT

The tremendous success of Deep Learning (DL) has significantly boosted the number of open-sourced DL frameworks hosted on GitHub. Among others, performance and accuracy bugs are critical factors that affect the reputation of these DL frameworks, therefore understanding the practice of discovering and investigating them for DL is important. In this paper, we conduct an exploratory study on the nature of reporting performance and accuracy bugs for DL frameworks, aiming to improve our knowledge on this topic. Our study covers 10 most popular open-sourced DL frameworks on GitHub (e.g., TensorFlow, Keras, and PyTorch), based on which we sample 664 representative performance and accuracy bug reports out of a total population of 22,522. Through systematic analysis, we found that: (1) low speed is the primary reason that a performance bug related report is submitted but we see no consistent pattern for accuracy related ones; (2) most of the reports are about issues encountered in the training stage; (3) only a small proportion of the reports provide insufficient information to investigate; (4) the majority of the performance and accuracy bug reports (from 69% to 100%) are not related to the actual bug or regarded as unclassified; (5) around 50% of the performance and accuracy bug reports, which indeed reveal bugs, are not resolved by direct patches. Deriving from the above, we discuss a set of actionable implications to the researchers, maintainers, and report submitters. To promote open science, the labeled dataset has been made publicly available at https://zenodo.org/record/6371676.

CCS CONCEPTS

• **Software and its engineering → Software creation and management**; **Software post-development issues**.

KEYWORDS

Empirical software engineering, mining software repositories, artificial intelligence, performance engineering

*Corresponding Author

EASE 2022, June 13–15, 2022, Gothenburg, Sweden

ACM ISBN 978-1-4503-9613-4/22/06…$15.00
https://doi.org/10.1145/3530019.3530029

ACM Reference Format:
Guoming Long and Tao Chen. 2022. On Reporting Performance and Accuracy Bugs for Deep Learning Frameworks: An Exploratory Study from GitHub. In *The International Conference on Evaluation and Assessment in Software Engineering 2022 (EASE 2022), June 13–15, 2022, Gothenburg, Sweden.* ACM, New York, NY, USA, 10 pages. https://doi.org/10.1145/3530019.3530029

1 INTRODUCTION

Deep learning (DL), which is a kind of machine intelligence algorithms that mimics the workings of the human brain in processing data [12], has been gaining momentum in both academia and industry [6–9, 21, 23]. Over the last decade, a variety of DL framework projects, such as TensorFlow and PyTorch, have been developed to enable rapid and seamless development of DL based software.

As with traditional software projects, DL frameworks inevitably contain bugs, especially those bugs that are related to performance (e.g., poor user experience, degraded responsiveness, and waste computational resources) [35] and accuracy (e.g., insufficient prediction outcomes and loss) [11]. Indeed, these performance and accuracy bugs in DL frameworks can lead to severe consequences, affecting any software that is built on top of them [13]. For example, according to the U.S. National Transportation Safety Board (NTSB), the recent accident of Uber's self-driving car was caused by performance and accuracy bugs of their DL framework, which inaccurately classified a pedestrian as an unknown object under specific conditions and doing so with a slow response[1]. Therefore, to improve the quality of continuous maintenance, mainstreamed DL frameworks make use of modern tracking systems — most commonly GitHub — to allow bugs to be reported, discussed, and eventually fixed. This paper focuses on understanding such a practice on the life-cycle of reporting performance and accuracy bugs for DL frameworks.

It has been well-recognized that the bug report analysis is at least as difficult as the actual bug-fixing [2, 15, 32]. In fact, Anvik et al. [3] discover that the developers and maintainers are often overwhelmed with a large number of bug reports. The task becomes even more time-consuming when it comes to understanding the content: Herzig et al. [16] report that it takes at least 90 working days of efforts (for two experienced developers) to merely classify around 7,000 reports — this does not even include extracting useful information from them. The reason could be partial because a bug report is often written in a considerable length, e.g., up to 332 sentences per report in average [25]. Furthermore, a submitted bug report may not be associated with an actual bug, which could only be known after inspection [16]. The analysis is particularly difficult for the performance and accuracy related bug reports since

[1]https://tinyurl.com/ykufbpey.

they are often implicit. That is, there is no precise oracle to assess them, meaning that it is typically hard to understand how "slow" or how "inaccurate" the results are would be considered as a bug without thorough investigation. As a result, insights on the state-of-the-practice for reporting bugs/concerns are as important as understanding the characteristics of an actual bug itself.

Previous work exists on understanding the causes of bugs and how they are fixed for traditional projects [22, 30], but there is a lack of studies that target explicitly performance and accuracy bugs. From another perspective, Zimmermann et al. [44] analyze the practice of how bugs are reported in traditional software from their classic tracking systems such as JIRA. However, since DL frameworks hold different stacks, fixed patterns, and software engineering practices from the traditional software projects [1, 19, 28], the conclusions drawn on traditional projects are not necessarily applicable to the DL ones. Further, most of the DL frameworks are open-sourced hosted on GitHub, whose tracking system is much more flexible, but highly unstructured compared with the classic ones. There exist studies that seek to investigate the characteristics of bugs for DL systems built on top of the DL frameworks [18, 29, 31, 39, 40]. However, they do not target the level of DL frameworks and there is still a lack of understanding on their bug reporting practice, particularly related to performance and accuracy concerns, which is our focus in this work.

To close such a gap, this work presents an exploratory study of 10 popular open-sourced DL frameworks from GitHub. In particular, we collect and analyze 664 high-quality representative samples of the performance and accuracy bug reports from a total population of 22,522. As such, we seek to provide a comprehensive understanding of five research questions related to the performance and accuracy bug reports for DL frameworks. Specifically, our key contributions are:

- Several empirical findings that provide better understandings of reporting performance and accuracy bugs for DL frameworks. In particular, we discuss answers and evidence for the following research questions (RQs):
  - **RQ1:** What is the most common reason for reporting?
    Answer: "low speed" is the most common reason for submitting performance related bug reports (from 27% to 67% among the frameworks); however, we see no consistent pattern for accuracy related ones.
  - **RQ2:** Which DL stage(s) is the most relevant in reporting?
    Answer: The *training* stage is prevalent in performance and accuracy bug reports, ranging between 38% to 77% across the frameworks.
  - **RQ3:** Do reports provide sufficient information for investigations?
    Answer: Yes, as states like "not related" or "not enough information" are rare cases.
  - **RQ4:** Are most reports bug-related?
    Answer: No, the majority (from 69% up to 100%) of the closed performance and accuracy bug reports are either unclassified or unrelated to actual bugs.
  - **RQ5:** Do bug-related reports always lead to patch(es)?
    Answer: In fact, around 50% of the performance and accuracy bug reports, which indeed reveal bugs, are not resolved by direct patches.
- Actionable implications to researchers, maintainers, and report submitters for the DL frameworks.
- A labeled dataset — a six months effort — that enables rapid proof-of-concept for future studies on problems related to performance and accuracy bug reports for DL. The dataset, together with other analyzed data, has been made publicly accessible via our online repository: https://zenodo.org/record/6371676.

The rest of this paper is organized as follows. In Section 2, we introduces the background information about performance and accuracy bug reports on GitHub. Section 3 describes the research methodology of our empirical study. Section 4 elaborates the detailed data preparation process, following by an articulation of the classification criteria in Section 5. Section 6 presents results and findings. The actionable implications derived from our findings are presented in Section 7. In Section 8 and 9, we discuss the threats to validity and related work, respectively. Finally, Section 10 draws a conclusion for this paper.

2 BACKGROUND

In this section, we introduce the necessary preliminaries for our empirical study.

2.1 Performance and accuracy Bug Reports for DL Frameworks

Indeed, despite being used primarily for bug reporting [29], the issues tracking system on GitHub can serve various purposes or even as a forum of discussion. However, those issues are formatted in a way that is intrinsically similar to the reports [5][2], including a title/summary, descriptions, comments, and labels. In this work, we are interested in analyzing those issues that are close to the "bug reports" in more traditional platforms, e.g., the JIRA. Specifically, our focus is the **performance and accuracy bug reports** — those submitted issues that report observations or concerns about the undesired phenomena on the performance and accuracy of the DL framework, which may reveal performance and accuracy bugs [18]. We will elaborate on the inclusion and exclusion criteria used to extract the performance and accuracy bug reports in Section 4.

It is worth noting that the performance and accuracy bug reports merely express **performance and accuracy concerns**, which do not necessarily correlate with actual **performance and accuracy bugs**. That is, it may well be possible that the reports turn out to be some false alarms (due to, e.g., incorrect usage of the API) or they report something that can be easily resolved by using certain workarounds, which do not require a patch to fix. Indeed, given the open nature of the issue tracking system, one can submit a performance or accuracy bug report as long as there is a performance or accuracy concern, regardless of how trivial it is. This, together with the fact that the bug report itself is complicated to be analyzed [2, 3, 15, 16, 25, 32], is the key reason why understanding the nature and life-cycle of performance and accuracy concerns/bugs

[2]Hence, whenever we call reports, we mean the issues on GitHub.

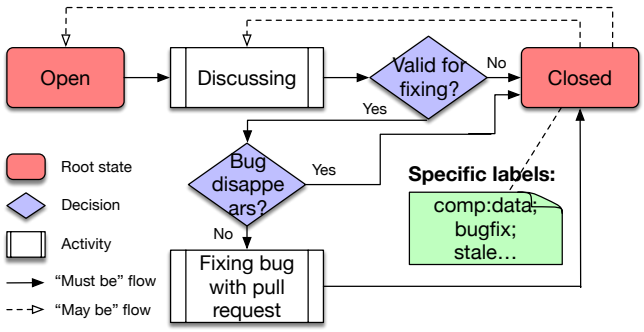

Figure 1: The brief lifecycle of bug reports on GitHub.

reporting for DL frameworks is crucial, which is our focus in this work.

2.2 Bug Reports Lifecycle on GitHub

On GitHub, all the bug reports (regardless of whether they are performance or accuracy specific) are committed to a standard lifecycle [34, 38]. As shown in Figure 1, a submitted bug report would undergo a discussion and commenting to identify whether it is valid for bug-fixing. If it does not involve a bug or cannot be determined, then the maintainers can close the report (with necessary conclusion and workarounds). If there is indeed a bug, a formal bug-fixing process would be triggered. During such a process, the bug may possibly disappear (e.g., being fixed unintentionally), in which case the report would be closed too. Otherwise, eventually, a directly associated patch(es) would be created with a pull request, awaiting approval of merge after which the report would be closed. Most commonly, a report is closed along with various custom labels added throughout the lifecycle of the bug report.

Note that the report may be reopened, as similar observations may occur again. Similarly, an extended discussion about the report is also possible even if the report has been closed.

3 METHODOLOGY

As shown in Figure 2, our research methodology consists of a *Data Preparation Phase* and a *Classification Phase*.

During *Data Preparation Phase*, we firstly conducted *Framework Selection* to extract the most popular DL frameworks on GitHub according to the number of stars/folks. Next, we retrieve a total number of 22,522 reports (including those related to other types of bugs) for all frameworks over the period of five years using PyGithub Module[3], for which is impractical to thoroughly analyze. Yet, according to the guidance from Kadam and Bhalerao [20], we need at least 664 samples of performance and accuracy bug reports to gain meaningful interpretation at 99% confidence level under such a population (see Section 4.2). Therefore, we randomly sampled from the 22,522 reports until the collected number of performance and accuracy bug reports reached 664 according to the *Inclusion and Exclusion Criteria*. For each framework, the sampled number is proportional to its percentage of the returned number of reports within the 22,522. This resulted in a manual inspection of more

[3]https://github.com/PyGithub/PyGithub

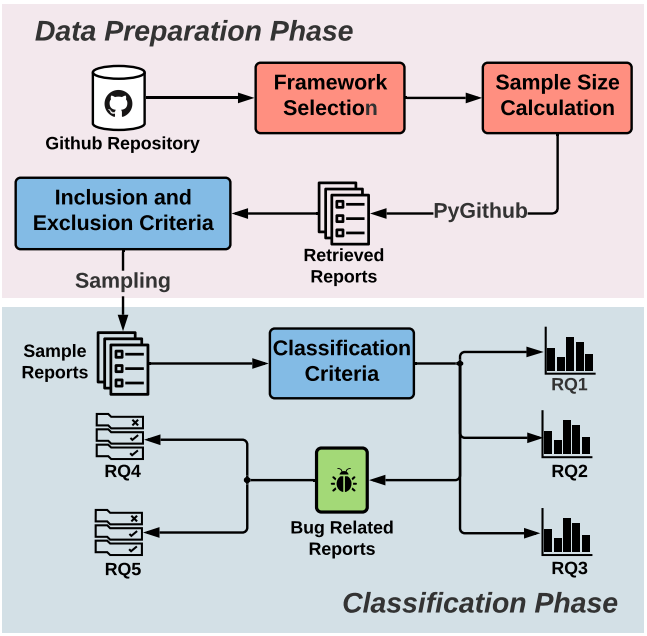

Figure 2: The empirical study methodology.

than 10,000 reports, including error-checking to ensure that none of the 664 performance and accuracy bug reports was misclassified, over the course of six months.

In the *Classification Phase*, we classified the 664 sampled performance and accuracy bug reports according to the defined *Classification Criteria*, which were derived from both the results of sampled reports and knowledge from prior work [18, 40] (see Section 5). To avoid bias, we ensure that the Cohen's Kappa coefficient (κ) [26] of the classification is at least 0.7 between the authors (which means a substantial agreement [26]). The results lead to our answers to **RQ1**-**RQ3**, together with the reports that indeed reveal performance and accuracy bugs. From those bug-related reports, we can then draw findings for **RQ4** and **RQ5**.

4 DATA PREPARATION

We conducted the data collection in Jan 2021. Here, we specify the detailed steps of the data preparation process.

4.1 Framework Selection

To select the DL frameworks for this study, we mined the most popular ones from Github (based on the number of stars and folks). The only criterion we used is that the framework should not tie to a specific application domain of DL. As such, popular frameworks like Theano, OpenCV, and Torch7 were omitted as they focus specifically on Computer Vision. We eventually chose 10 most widely used DL frameworks, as shown in Table 1.

4.2 Sampling Method and Size

Using the PyGithub module for all 10 DL frameworks on GitHub, we retrieve a total number of 22,522 reports submitted between 1st Jan 2014 and 31st Dec 2019. We chose this period as it contains

a more balanced number of open and closed reports. Since this is an extremely large number of reports, we wish to sample a set of high-quality representatives, ensuring that our conclusions would generalize to the whole population of each framework. To that end, we calculate the proper sample size following the guidance offered by Kadam and Bhalerao [20]:

$$\frac{N \times \varphi}{N + \varphi}, \quad \text{subject to} \quad \varphi = \frac{z^2 \times p \times (1-p)}{e^2} \tag{1}$$

where N is the total number of reports (i.e., 22,522); z is the two-sided z-score at a confidence level of 99%; e is the corresponding margin of error and p is the proportion of performance and accuracy bug reports in the entire population (using the most conservative value 0.5). The above has led to 664 as the total sample amount for performance and accuracy bug reports. Then, we proportionally and randomly sample the performance and accuracy bug reports in different frameworks and states, according to the ratio between its total number for a framework/state and the total amount of searched reports for all frameworks, as shown in Table 1. This is important as there is an imbalanced distribution of the number of reports across the DL frameworks. Note that during the process, we ensure that the selection is completely random — every report will have an equal chance to be selected. Again, all authors are involved in the process to improve reliability. In case of disagreement, the reports were investigated multiple times or counseling external experts until a consensus has been reached.

4.3 Inclusion and Exclusion Criteria

When sampling reports, we use the following inclusion criteria to decide whether a report should be considered as a performance or accuracy bug report:

- The report contains at least one clear symptom of performance or accuracy concern, such as the hang, unexpected loss, and slow speed (see next section).
- The report describes an observation, concern, or problem about the undesired phenomena on the performance or accuracy aspects of using the DL framework.
- The report has at least one label (in addition to open/closed).

We remove the report if it fits any of the exclusion criteria:

- The report is related to documentation or a tutorial.
- It is a request for completely new features, despite being performance or accuracy-related.
- It is a thread of pure discussion, user feedback, or records of planned "TODO" tasks.
- It describes an error that causes a crash when using the DL framework.

5 CLASSIFICATION AND LABELING

In this section, we present the criteria used to classify and label the sampled reports, which is the foundation of this study to derive and analyze our findings.

5.1 Classification Criteria

5.1.1 Symptoms Claimed in DL Performance and Accuracy Bug Reports. To better study the reports, we summarize the following common symptoms of performance and accuracy for DL framework as inspired by the work of Zhang et al. [40]. In particular, the performance related symptoms are:

- **Low Speed:** DL processing speed is rather slow in certain pipeline stages (e.g., training and prediction etc).
- **Abnormal Memory Usage:** The process consumes an abnormal amount of memory, e.g., too low, too high, or a possible leak. The case that leads to a crash is not included.
- **Hang:** The DL process is not responsive or runs indefinitely.

In contrast, the symptoms for accuracy are:

- **Poor Loss:** Abnormal observation of the loss value during training, e.g., high loss, infinite loss, periodic loss, unchanged loss, and other unwanted loss values.
- **Poor Prediction:** Poor metric result during prediction (e.g., precision, recall, or accuracy) is observed.
- **Unexpected Output:** This occurs when the output of DL system contains, e.g., abnormal gradient value, abnormal weight value, unusual tensor calculation result.

Table 1: Distribution of 664 samples for each DL project/state.

Framework	Retrieved Open	Retrieved Closed	Retrieved Total	Sampled Open	Sampled Closed	Sampled Total
TensorFlow	1657	7814	9471	49	230	279
Keras	1392	3215	4607	41	94	135
PyTorch	1123	2094	3217	33	62	95
MXNet	445	1771	2216	13	52	65
Caffe	178	957	1135	5	28	33
CNTK	187	506	693	6	15	21
Chainer	2	424	426	0	13	13
Darknet	341	83	424	10	3	13
Caffe2	106	93	199	3	3	6
Tiny-dnn	66	68	134	2	2	4
Total	5497	17025	22522	162	502	664

5.1.2 Stages of the DL Pipeline. In this work, we classify the performance and accuracy bug reports into the following five key stages of typical DL systems when the frameworks are used to build them, as recommended by Islam et al. [18]:

- **Data Processing:** This stage is responsible for data loading, preprocessing, input, and output filtering that enable more effective model training.
- **Model Construction:** This is related to the choice of model and hyperparameter tuning[4].
- **Training:** This involves training a neural network such that a loss function is minimized across data samples. It is the core of DL pipeline and can be strongly tied with hardware resources, e.g., GPU and CPU parallelism.
- **Evaluation:** This is concerned with validating and evaluating the quality of the model trained. Common metrics such as AUC, F-measure, or RMSE would be used here.
- **Prediction:** This is the stage where the DL system with a trained model is actually deployed in production, forecasting the outcome with newly given data.

[4]Unlike Islam et al. [18], we consider hyperparameter tuning as part of Model Construction as we found that they are often mentioned together in a performance or accuracy bug report.

5.1.3 States of Performance and Accuracy Bug Reports. We found that most of the custom labels in GitHub are not states; even for those which indeed represent states, the majority of them are duplicate or represent similar meaning. Therefore, in what follow we summarize 11 states across the DL frameworks under which the report is closed:

- **Fixed:** The report where a related patch is directly created or there is a claim that the bug is fixed in another release.
- **Resolved:** The state where an accepted workaround has been provided or the submitter discovers an alternative resolution. However, no change needs to be made to the codebase.
- **Not reproducible:** This means the reported observation has been found as difficult to be reproduced.
- **Not enough information:** This is often a closed report where the provided information has been claimed as too vague to generate discussion and investigation.
- **Not related:** The report has been confirmed to be unrelated to the DL framework.
- **No reason:** The submitter closes the report without giving any comment.
- **Better ask in elsewhere:** The maintainers suggest that the report is not suitable to be discussed on GitHub.
- **Working as expected:** The report is identified as not a concern, but merely about DL code design.
- **Lack of activity:** Closed by issue management system due to being idle over a period of time.
- **Stale:** The report has been identified by a maintainer as stale, hence should be closed.
- **Duplicate:** The report is closed as the same content has already been reported in an origin one.

In contrast, there is only one state to represent open reports:

- **Open:** The performance and accuracy bug report has yet reached a conclusion about the next stage.

5.1.4 Correlation between Reports and Bugs. Drawing on the states, we are able to easily summarize the performance and accuracy bug reports into the three categories below[5]:

- **Bug-related:** The reports under the state of *fixed* are confirmed related to performance and accuracy bug by the maintainers, as it indicates that the reported observation has revealed a bug that triggers a bug-fixing process.
- **Bug-unrelated:** The reports are closed without associated fixes while having the states of *resolved*, *not reproducible*, *not enough information*, *not related*, *better ask in elsewhere*, *working as expected* are regarded as unrelated to performance and accuracy bugs by the maintainers, since they do not reveal any actual bugs.
- **Unclassified:** *No reason*, *lack of activity*, and *stale* states mean a report provides no information for maintainers to determine whether a bug is involved or not.

For those bug-related reports, we further classify them depending on how the corresponding performance and accuracy bug is fixed:

- **Fixed by patch(es):** This means that the bug is fixed by directly merging a patch(es) into the codebase.

[5]For the very small amount of reports with a *duplicate* state, we use the state of its master bug report.

Table 2: % on the reasons of submitting performance and accuracy bug reports for the DL frameworks (the most common one is highlighted). Note that a small amount of reports are linked with more than one reasons.

Frameworks	Performance	Accuracy	Low Speed	Abnormal Memory Usage	Hang	Poor Loss	Unexpected Output	Poor Prediction
Tensorflow	73%	27%	52%	14%	6%	5%	13%	9%
Keras	38%	64%	32%	5%	1%	25%	8%	31%
PyTorch	71%	29%	44%	15%	12%	11%	17%	2%
MXNet	62%	40%	51%	8%	3%	5%	6%	29%
Caffe	39%	61%	27%	6%	6%	24%	21%	15%
CNTK	64%	32%	45%	18%	0%	5%	23%	5%
Chainer	69%	31%	62%	8%	0%	8%	15%	8%
Darknet	15%	85%	15%	0%	0%	38%	15%	31%
Caffe2	67%	33%	67%	0%	0%	0%	33%	0%
Tiny-dnn	25%	75%	25%	0%	0%	0%	25%	50%

- **Fixed in newer release:** This refers to the report where there is no directly associated patch(es), but it was commented that the bug disappears in a newer release, implying that it must have been indirectly fixed as part of some other patches or refactoring. Note that this implies that the reported performance and accuracy concerns were fixed even without being raised by a report.

All authors of this paper labeled the reports based on the classification criteria above by interpreting the content and comments of each report. Disagreements were resolved internally or by counseling external experts when needed. From these, we achieve a Cohen's Kappa coefficient $\kappa \in [0.7, 1.0]$ in each corresponding criteria after labeling, which indicates a substantial agreement [26].

6 RESULTS

In this section, we present the results of our empirical study and answer the research questions posed in Section 1. Note that to avoid bias, we analyze each DL framework individually and draw conclusions therein, since they have different total numbers of reports. The dataset and raw data are made available at: https://zenodo.org/record/6371676.

6.1 Reasons of Reporting (RQ1)

As can be seen from Table 2 (two left-most columns), the concerns over performance and accuracy tend to be balanced across the DL frameworks. Looking into more detailed reasons of both performance or accuracy related bug reports, we see that for a majority of cases, the developers submit performance and accuracy bug reports mainly due to *low speed* (8 out of 10, up to 67%), especially

Table 3: % on the relevant DL stages to the submitted performance and accuracy bug reports (the most common one is highlighted). Note that a small amount of reports are linked with more than one stage.

Framework	Data Processing	Model Construction	Training	Evaluation	Prediction
Tensorflow	18%	22%	61%	13%	17%
Keras	5%	30%	44%	16%	11%
PyTorch	33%	43%	52%	37%	24%
MXNet	14%	18%	65%	14%	14%
Caffe	3%	36%	48%	6%	6%
CNTK	10%	24%	38%	24%	5%
Chainer	15%	23%	69%	8%	15%
Darknet	15%	0%	77%	8%	0%
Caffe2	0%	17%	33%	0%	50%
Tiny-dnn	0%	0%	0%	50%	50%

for popular frameworks such as TensorFlow, MXNet, Chainer, and Caffe2. This is surprising, as despite the accuracy is a unique and key attribute for DL frameworks, the primary concern remains on the performance, i.e., time-related attributes.

Another observation is that, for the three concrete reasons under the performance concern, there is a strong bias towards the symptom of *low speed*. In contrast, the concrete reasons of accuracy concern are relatively more balance and we cannot conclude which one is more prevalent.

Compared with the others, we found that to what extent can be considered as low speed in a performance bug report of DL frameworks is often more vaguely defined and with significantly different aspirations based on the context. For example, report `#3996` for Keras reports that the "`Model.fit takes about 10 minutes before it actually starts doing anything`", upon which is considered unacceptable. In contrast, report `#33340` for TensorFlow states that a prediction speed of 29ms is already a major slowdown. Such vagueness and diverse aspirations could explain why *low speed* is the main reason for submitting a performance or accuracy bug report.

Therefore we say:

Finding 1: *For DL frameworks, low speed is the most prevalent concrete reason for submitting a performance related bug report (from 27% up to 67%). For accuracy related ones, we see no definitive patterns on the reason.*

6.2 Reported Learning Stages (RQ2)

From Table 3, it is clear that all the key stages in DL are associated with the performance and accuracy bug reports. However, the majority of them are related to the *training* stage of the DL pipeline for 8 out of 10 frameworks, all of which are the top most popular DL frameworks we found. We also note that the *data processing* and *model construction* tend to be two of the most uncommon stages in the performance and accuracy bug reports submitted.

We found that the performance and accuracy bug reports related to *training* do not only predominate, but also lead to some of the most serious consequences. For example, report `#9873` on PyTorch reports that "`PyTorch is slow when only using CPU, and cannot utilize multicore of CPU`", causing it to become about 30% slower than Keras to train under the same condition.

In summary, we conclude that:

Finding 2: *The training stage is significantly more predominately concerned and relevant than the remaining four stages as stated in the performance and accuracy bug reports, constituting between 38% and 77%.*

Table 4: % on the mutually exclusive states of performance and accuracy bug reports in DL projects (the most common one is highlighted).

Frameworks	Fixed	Resolved	Not reproducible	Not enough information	Not related	No reason	Better ask in elsewhere	Working as expected	Lack of activity	Stale	Duplicate	Open
Tensorflow	12%	29%	1%	2%	2%	10%	4%	5%	14%	1%	3%	18%
Keras	4%	30%	0%	0%	1%	2%	0%	8%	2%	21%	1%	30%
PyTorch	20%	19%	0%	0%	1%	5%	16%	1%	1%	1%	2%	34%
MXNet	20%	32%	0%	0%	0%	0%	2%	8%	17%	2%	0%	20%
Caffe	12%	18%	6%	3%	3%	36%	0%	6%	0%	0%	0%	15%
CNTK	14%	38%	0%	0%	0%	0%	0%	10%	5%	0%	0%	33%
Chainer	31%	23%	0%	0%	8%	8%	0%	8%	0%	23%	0%	0%
Darknet	0%	15%	0%	0%	0%	0%	0%	8%	0%	0%	0%	77%
Caffe2	0%	33%	0%	0%	0%	0%	17%	0%	0%	0%	0%	50%
Tiny-dnn	0%	25%	0%	0%	0%	0%	0%	0%	25%	0%	0%	50%

6.3 Report States (RQ3)

In Table 4, most commonly a performance or accuracy bug report is under an *open* or *resolved* state, as they are the most (or second most) prevalent for 7 frameworks. Note that a *resolved* is different from a *fixed*, as the former does not trigger a process of bug fixing; in most cases, the report is resolved because a workaround is provided; or there is a claim that the observation does not exist anymore. For example, report `#9026` on MXNet reports that training speed is extremely slow with NVIDIA V100 GPU. Further investigation suggested a workaround of using `DataLodear` that can feed data asynchronously instead, which then "resolves" bug report but there is not a "fix", since no actual patch has been generated.

In particular, PyTorch and MXNet exhibit good balance on *open*, *fixed*, and *resolved* state with high parentage, implying a particularly healthy "report-then-address" cycle on performance and accuracy -related concerns. Darknet, Caffe2, and Tiny-dnn have a much higher share on *open* than *fixed* and *resolved*, suggesting inactive maintenance, despite they are used in practice. Chainer, on the other extreme, has no open performance or accuracy bug reports, suggesting it is either doing extremely well or simply attracts only a rather small proportion of users.

Note that *not related*, *not enough information*, *duplicate*, and *not reproducible* are rare states, suggesting a good sign that the performance and accuracy bug reports submitted are of high quality.

Therefore, we say:

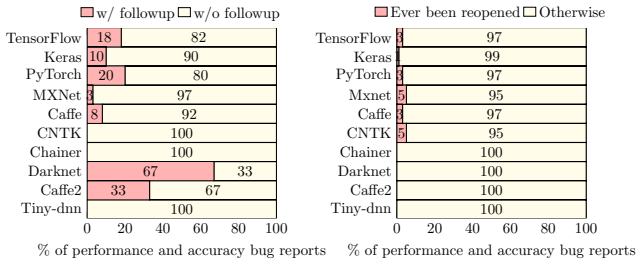

Figure 3: % of closed performance and accuracy bug reports with/without followups (left) and whether they have ever been reopened (right).

Finding 3: *The performance and accuracy bug reports under the state of "open" and "resolved" are significantly more prevalent than the others, suggesting a healthy maintenance cycle of performance and accuracy concerns in DL frameworks.*

Finding 4: *In contrast, the reports in the state of "not related", "not enough information", "duplicate", and "not reproducible" are much more rare, meaning that the performance and accuracy bug reports often provide sufficient information in DL frameworks.*

6.3.1 Reopened reports and followups on closed reports. Two related and interesting questions to answer are what happens after a report is closed under whatever specific state, and how common for a closed report to be reopened? To this end, we look at whether a closed performance or accuracy bug report can still have a sustainable discussion and whether (non-trivially) reopening reports are common[6].

From Figure 3 (left), we see that mostly the discussion of a performance or accuracy bug report ends once the report is closed. However, there is also a good amount of them (up to 67% on Darknet) where the new observations and discussion still continues without reopening them, since the proportions of the reports that have been ever reopened is significantly lower as shown in Figure 3 (right). Yet, despite rarely resulting in a reopening, such an extended discussion implies the importance/prevalence of the reported performance and accuracy observations/concerns, and then the report was closed without full satisfaction. In fact, the followups often lead to very positive outcomes. For example, in report `#31243` for TensorFlow, it was reported that `tf.keras.load_model` is very slow, the resolution proposed before closing the report is by installing `tf-nightly-gpu-2.0-preview`, which however poses some compatibility issues. In the extended dissuasion after the report was closed, a participant confirmed that the compatibility can cause a non-trivial issue, and a simpler workaround that loads the model from the function each time was suggested (by a different participant) and accepted.

Thus, we conclude that:

Finding 5: *For DL frameworks, it is not uncommon that follow-up observations/discussions are made to an already closed performance or accuracy bug report. However, reopening a report is rare.*

[6]We count each report exactly once even if it has multiple reopens and followups on multiple closed states.

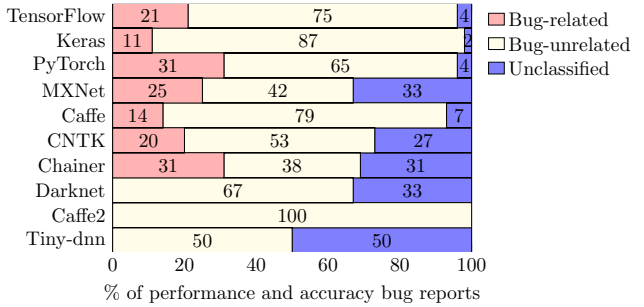

Figure 4: % of closed performance and accuracy bug reports that are bug-related, bug-unrelated, or unclassified.

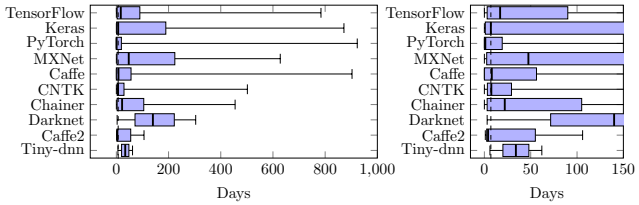

Figure 5: Distribution of the time required (in days) for a performance or accuracy bug report to be closed (dashed line denotes a week). The left shows an overall picture and the right is the "zoom in" version with a finer-grained scale.

6.4 Bug Revealing Reports (RQ4)

For all the performance and accuracy bug reports that were closed in DL frameworks, Figure 4 shows how many of them can actually reveal at least one performance or accuracy bug. Surprisingly, only small proportions of them are bug-related (between 11% to 31%). In contrast, it is most prevalent that a performance or accuracy bug report is bug-unrelated (from 42% up to 100%), with those considered as unclassified ranked as the second most common. In particular, the number of unclassified reports is rare for TensorFlow, Keras, PyTorch, and Caffe2, while their proportions of bug-unrelated ones remain very high (with 100% for Caffe2).

The above is a surprising sign that, albeit the performance and accuracy bug reports themselves in DL frameworks are generally of good quality, they do not often reveal actual performance and accuracy bugs that require bug-fixing.

From the above, we can summarize that:

Finding 6: *In DL frameworks, the majority (from 69% up to 100%) of the closed performance and accuracy bug reports are either unrelated to actual performance and accuracy bugs (i.e., bug-unrelated) or unclassified.*

6.4.1 Time required. A related question is how much time would be required to identify whether a performance or accuracy bug report indeed reveals a bug? To this end, we further analyze, for all closed ones, the time (in days) taken between the reports being submitted and the comment that confirms its bug-relevance, i.e., whether it indeed discloses a performance or accuracy bug.

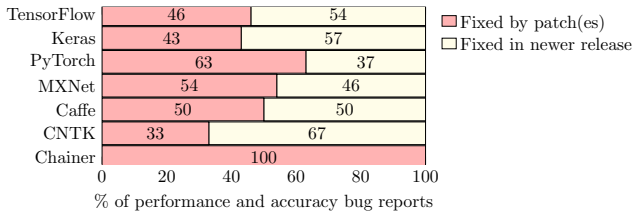

Figure 6: % of fixed performance and accuracy bug reports that are fixed directly by patch(es) or indirectly in newer releases (Tiny-dnn, Caffe2, and Darknet are omitted as they have no fixed reports in our samples).

Figure 5 illustrates the results, in which we see that, for the median on 8 out of 10 frameworks, it needs a week or more for concluding whether the report is valid for bug-fixing. In particular, most of them exhibit a rather high interquartile range (e.g., Keras, MXNet, and Darknet), with the 75th percentile being more than 100 days. This suggests that the amount of time required (and potentially the efforts) is considerably high in general across the DL frameworks.

Therefore, we say:

Finding 7: *For DL frameworks, usually it takes at least a week to identify whether a performance or accuracy bug report indeed reveals a bug.*

6.5 Patch(es) on Reports (RQ5)

We seek to examine the distribution of whether a fix in a bug-related report is completed with a direct patch(es) or has already been dealt with in newer releases (as stated by the comments from the reports). As such, we consider only the reports under *fixed* state. We do not consider those reports with a *duplicate* state, as in those cases a performance or accuracy bug is fixed when raised by another report. Therefore, when it is stated that a bug is fixed in a newer release, it often means that the bug was detected by the developer during routine refactoring rather than being raised from a report. As can be seen from Figure 6, how a bug-related report is handled exhibits a reasonable balance between the two categories across all DL frameworks (except for Chainer). This is to our surprise, as it suggests that around half of the performance and accuracy bugs on DL frameworks were fixed without being formally raised in a report.

Finding 8: *Around half of the performance and accuracy bug reports in DL frameworks, which indeed reveal bugs, do not associate with a direct patch.*

7 ACTIONABLE IMPLICATIONS

We now discuss what actionable implications our empirical study and findings can provide to the practitioners of bug report analysis for DL frameworks.

7.1 To Researchers

As a first step, our empirical study provides clear motivations and the necessary data for researchers to investigate a wide range of related research problems on analyzing performance and accuracy bug reports for DL frameworks. In particular, our labeled dataset serves as the readily available foundation to efficiently provide proof-of-concept. Specifically, we provide the following implications:

(1) **Finding 5** reveals that discussion on a performance or accuracy concern continues even after the report has been closed. However, little has been done to properly reflect the value and result of such discussion on the state of a report. It calls for future research to consider formalizing a more systematic protocol, or automated tools, that helps to make the decision on closing and reopening a performance or accuracy bug report for DL frameworks.
(2) **Finding 6** shows that only a small proportion of the performance and accuracy bug reports are revealing performance and accuracy bugs. In the meantime, from **Finding 7**, we note that it often takes a considerably long time to identify whether a performance or accuracy bug report can indeed reveal bugs. It is, therefore, desirable to have an automatic predictor that can identify which bug reports are worth bug-fixing, thus saving a significant amount of maintenance efforts.

7.2 To Maintainers

Deriving on the findings, we can provide the following actionable recommendations to the maintainers of the DL frameworks:

(1) **Finding 1** suggests that greater effort is required to maintain, improve, or more correctly guide the users to achieve the best training and prediction speed for DL frameworks.
(2) Since **Finding 2** reveals that *training* is the more concerned and relevant DL stage in a performance or accuracy bug report, hence the documentation or code related to *training* requires more attention to maintain for performance and accuracy reasons. The goal is to reduce the change of bug reports in the first place, regardless of whether they are indeed revealing bugs or merely false alarms.

7.3 To Report Submitters

Our findings also draw actionable suggestions to the report submitters for the DL frameworks:

(1) **Finding 3** and **Finding 4** suggest that the current practice of writing performance and accuracy bug reports is healthy, therefore our results confirm no demand of any significant change.
(2) **Finding 8** reveals that, for DL frameworks, around half of the performance and accuracy bug reports that can indeed reveal bugs do not lead to direct patches. Therefore, we suggest that before submitting a performance or accuracy related report, one should also examine the version at the newest "release candidate" branch, even if that is not a stable version. This would likely help reduce the bug-related reports that do not lead to an actual patch.

8 THREATS TO VALIDITY

Threats to **internal validity** can be related to the classification of the performance and accuracy bug reports. We mitigate this in two steps: firstly, we codify the classification criteria based on either what has been well-acknowledged [40] or deriving from our samples, which involves all authors in order to reach common agreements. The identification of whether a report is a performance or accuracy bug report also follows systematic inclusion and exclusion criteria, which is only confirmed once agreed by all authors. Secondly, when labeling the performance and accuracy bug reports, the inter-rater agreements were measured using Kappa coefficient (κ). From this, we achieve $\kappa \geq 0.7$ in all cases. However, we admit that errors may be inevitable during the manual process.

To ensure **construct validity** and avoid **conclusion bias** to particular DL frameworks, we consider the total number of reports for each (and those with the open and closed root state), we then conduct a random sample proportionally according to their totals. This helps us to better ensure a fair comparison under the imbalance distributions between frameworks and root states. In particular, upon reporting the results, we leverage the % of individual frameworks whenever required, which further prevents the conclusions from being dominated by certain DL frameworks.

The other threats can be related to the **external validity**, which is about the trustworthiness and generalizability of the conclusion drawn from the samples. We tackle this by investigating 10 DL frameworks with diverse characteristics and scales. For the actual sampling, we follow what has been recommended by Kadam and Bhalerao [20] to calculate the required sample size, which ensures that we are 99% confident that the samples are representative enough for the whole population. Indeed, examining more DL frameworks and samples may provide more insights, but this is a rather expensive process as it has already taken around six months to analyze what we have collected in this paper.

9 RELATED WORK

We now discuss the prior work in light of the purpose and findings of our empirical study.

Studies on Bugs for Projects based on Learning Frameworks: Since the modern era of Artificial Intelligence, there has been a few studies focusing on the characteristics of real bugs on projects based on learning frameworks [18, 29, 31, 39, 40]. Among them, Thung et al. [31] and Zhang et al. [39] focus on traditional bug report repositories, while Sun et al. [29], Islam et al. [18], Humbatova et al. [17] and Zhang et al. [40] work on data from GitHub commits/issues. Nevertheless, our work differs from the above on that:

- Instead of analyzing the characteristics of the real bugs (as in the above work), we focus on understanding the practice of reporting performance and accuracy bugs, i.e., the performance and accuracy bug report itself, from GitHub.
- Our purpose is to study the reports about performance and accuracy bugs as opposed to the general bugs (where most commonly the functional ones are the majority) from prior work. This allows us to draw more specific conclusions.
- We target the level of DL framework as opposed to the software that is built on top of it.
- We collect and analyze a moderate size of 664 samples from 10 DL frameworks.

Studies on Performance and Accuracy Bugs: While accuracy bugs are rarely studied for traditional software projects, performance bugs have been shown to be more critical and difficult to deal with compared with their functional counterparts, c.f. [14, 37]. For general open-sourced projects, a number of studies have been conducted to understand the cause, severity, and possible fix of real performance bugs [10, 33, 41, 42], ranging from 109 to 700 samples, from classic bug repositories like JIRA.

Studies of performance bugs on a specific domain of projects also exist. For example, Liu et al. [24] investigate 70 performance bugs collected from eight Android projects. The results cover properties such as impacts, bug manifestation, debugging, and bug-fixing effort. Likewise, Selakovic and Pradel [27] analyze 98 performance bugs from 16 client-side and server-side JavaScript projects. However, we focus on different purposes from those studies:

- We investigate not only performance bug reports but also accuracy bug reports, which are rarely studied.
- Our empirical study focuses on the nature of performance and accuracy bug reports rather than the bugs themselves.
- We focus on DL frameworks, which have been shown that share little similarity to traditional projects [1, 19, 28].
- We rely on GitHub that allows bugs to be reported without complying with more restricted rules compared with, e.g., JIRA. This imposes more difficulty in analysis.

Studies on Bug Reports: Empirical studies focusing on the bug report itself have also been an important thread of research. With the target of traditional software projects, Zimmermann et al. [44] investigate what criteria can form a high-quality bug report that is most useful for bug-fixing. Xia et al. [36] also empirically study how the bug reports are labeled, assigned, and given states. Zhao et al. [43] seek to understand whether there is a correlation between the discussion on a bug report and the quality of bug-fixing. Finally, by studying the Android-based projects, Bhattacharya et al. [4] focus on how the quality of bug reports on different types of bugs can impact the developers' behavior. However, the above work did not target performance and accuracy bug reports for DL frameworks.

10 CONCLUSION AND FUTURE WORK

In this work, we perform an empirical study that seeks to better understand the practice of reporting performance and accuracy bugs for DL frameworks. Our study systematically samples and analyzes 664 performance and accuracy bug reports from 22,522 issues over 10 DL frameworks on GitHub. The key findings are:

- "low speed" is the key reason for submitting performance related bug reports while the reason for reporting accuracy related concerns varies.
- *training* is the most prevalent DL stage in the performance and accuracy bug report.
- the performance and accuracy bug reports are often of sufficient information.
- majority of the performance and accuracy bug reports do not reveal actual bugs.
- around half of the performance and accuracy bug reports, which reveal bugs, do not associate with the direct patches.

Drawing on the findings, we provide actionable implications to researchers, maintainers, and submitters involved in the performance and accuracy bug reporting process for DL frameworks.

With this paper, we hope to raise the importance of understanding the reporting practice of performance and accuracy bugs for DL frameworks. Indeed, by leveraging the dataset from this work, the aforementioned implications have also hinted at the necessity of possible future research threads, such as automatic tools on better performance and accuracy bug report identification.

REFERENCES

[1] Saleema Amershi, Andrew Begel, Christian Bird, Robert DeLine, Harald C. Gall, Ece Kamar, Nachiappan Nagappan, Besmira Nushi, and Thomas Zimmermann. 2019. Software engineering for machine learning: a case study. In *Proc. 41st International Conference on Software Engineering: Software Engineering in Practice*.

[2] Giuliano Antoniol, Kamel Ayari, Massimiliano Di Penta, Foutse Khomh, and Yann-Gaël Guéhéneuc. 2018. Is it a bug or an enhancement?: a text-based approach to classify change requests. In *Proc. 28th Annual International Conference on Computer Science and Software Engineering, CASCON 2018*. ACM.

[3] John Anvik, Lyndon Hiew, and Gail C. Murphy. 2005. Coping with an open bug repository. In *Proc. 2005 OOPSLA workshop on Eclipse Technology eXchange*.

[4] Pamela Bhattacharya, Liudmila Ulanova, Iulian Neamtiu, and Sai Charan Koduru. 2013. An Empirical Analysis of Bug Reports and Bug Fixing in Open Source Android Apps. In *17th Conference on Software Maintenance and Reengineering*.

[5] Tegawendé F. Bissyandé, David Lo, Lingxiao Jiang, Laurent Réveillère, Jacques Klein, and Yves Le Traon. 2013. Got issues? Who cares about it? A large scale investigation of issue trackers from GitHub. In *IEEE 24th International Symposium on Software Reliability Engineering, ISSRE 2013*.

[6] Tao Chen. 2019. All versus one: an empirical comparison on retrained and incremental machine learning for modeling performance of adaptable software. In *Proceedings of the 14th International Symposium on Software Engineering for Adaptive and Self-Managing Systems*. 157–168.

[7] Tao Chen and Rami Bahsoon. 2017. Self-Adaptive and Online QoS Modeling for Cloud-Based Software Services. *IEEE Trans. Software Eng.* 43, 5 (2017), 453–475.

[8] Tao Chen and Rami Bahsoon. 2017. Self-Adaptive Trade-off Decision Making for Autoscaling Cloud-Based Services. *IEEE Trans. Serv. Comput.* 10, 4 (2017), 618–632.

[9] Tao Chen, Ke Li, Rami Bahsoon, and Xin Yao. 2018. FEMOSAA: Feature-Guided and Knee-Driven Multi-Objective Optimization for Self-Adaptive Software. *ACM Trans. Softw. Eng. Methodol.* 27, 2 (2018), 5:1–5:50.

[10] Yiqun Chen, Stefan Winter, and Neeraj Suri. 2019. Inferring Performance Bug Patterns from Developer Commits. In *30th IEEE International Symposium on Software Reliability Engineering, ISSRE 2019*.

[11] Anthony Di Franco, Hui Guo, and Cindy Rubio-González. [n.d.]. A comprehensive study of real-world numerical bug characteristics. In *Proceedings of the 32nd IEEE/ACM International Conference on Automated Software Engineering*. 509–519.

[12] Ian Goodfellow, Yoshua Bengio, Aaron Courville, and Yoshua Bengio. 2016. *Deep learning*. Vol. 1. MIT press Cambridge.

[13] Qianyu Guo, Xiaofei Xie, Yi Li, Xiaoyu Zhang, Yang Liu, Xiaohong Li, and Chao Shen. 2020. Audee: Automated Testing for Deep Learning Frameworks. In *35th IEEE/ACM International Conference on Automated Software Engineering, ASE 2020, Melbourne, Australia, September 21-25, 2020*. IEEE, 486–498.

[14] Xue Han and Tingting Yu. 2016. An Empirical Study on Performance Bugs for Highly Configurable Software Systems. In *Proc. 10th ACM/IEEE International Symposium on Empirical Software Engineering and Measurement, ESEM'16*. ACM.

[15] Xue Han, Tingting Yu, and David Lo. 2018. PerfLearner: learning from bug reports to understand and generate performance test frames. In *Proc. 33rd ACM/IEEE International Conference on Automated Software Engineering, ASE 2018*. ACM.

[16] Kim Herzig, Sascha Just, and Andreas Zeller. 2014. It's not a bug, it's a feature: how misclassification impacts bug prediction. In *35th International Conference on Software Engineering, ICSE 2013*. IEEE Computer Society.

[17] Nargiz Humbatova, Gunel Jahangirova, Gabriele Bavota, Vincenzo Riccio, Andrea Stocco, and Paolo Tonella. 2020. Taxonomy of real faults in deep learning systems. In *ICSE 2020: 42nd International Conference on Software Engineering*. ACM.

[18] Md Johirul Islam, Giang Nguyen, Rangeet Pan, and Hridesh Rajan. 2019. A comprehensive study on deep learning bug characteristics. In *Proc. ACM Conference and Symposium on the Foundations of Software Engineering, FSE 2019*. ACM.

[19] Md Johirul Islam, Rangeet Pan, Giang Nguyen, and Hridesh Rajan. 2020. Repairing deep neural networks: fix patterns and challenges. In *ICSE 2020: 42nd International Conference on Software Engineering*. ACM.

[20] Prashant Kadam and Supriya Bhalerao. 2010. Sample size calculation. *International journal of Ayurveda research* 1, 1 (2010).

[21] Alex Krizhevsky, Ilya Sutskever, and Geoffrey E. Hinton. 2012. ImageNet Classification with Deep Convolutional Neural Networks. In *26th Annual Conference on Neural Information Processing Systems 2012*.

[22] Zhenmin Li, Lin Tan, Xuanhui Wang, Shan Lu, Yuanyuan Zhou, and Chengxiang Zhai. 2006. Have things changed now?: an empirical study of bug characteristics in modern open source software. In *Proc. 1st Workshop on Architectural and System Support for Improving Software Dependability, ASID 2006*. ACM.

[23] Muyang Liu, Ke Li, and Tao Chen. 2020. DeepSQLi: deep semantic learning for testing SQL injection. In *ISSTA '20: 29th International Symposium on Software Testing and Analysis, Virtual Event, USA, July 18-22, 2020*. 286–297.

[24] Yepang Liu, Chang Xu, and Shing-Chi Cheung. 2014. Characterizing and detecting performance bugs for smartphone applications. In *36th International Conference on Software Engineering, ICSE 2014*. ACM.

[25] Senthil Mani, Rose Catherine, Vibha Singhal Sinha, and Avinava Dubey. 2012. AUSUM: approach for unsupervised bug report summarization. In *20th ACM SIGSOFT Symposium on the Foundations of Software Engineering*. ACM.

[26] Mary L McHugh. 2012. Interrater reliability: the kappa statistic. *Biochemia medica* 22, 3 (2012).

[27] Marija Selakovic and Michael Pradel. 2017. Performance Issues and Optimizations in JavaScript: An Empirical Study. In *Software Engineering 2017*.

[28] Siwakorn Srisakaokul, Zhengkai Wu, Angello Astorga, Oreoluwa Alebiosu, and Tao Xie. 2018. Multiple-Implementation Testing of Supervised Learning Software. In *The Workshops of the The Thirty-Second AAAI Conference on Artificial Intelligence, 2018 (AAAI Workshops, Vol. WS-18)*. AAAI Press.

[29] Xiaobing Sun, Tianchi Zhou, Gengjie Li, Jiajun Hu, Hui Yang, and Bin Li. 2017. An Empirical Study on Real Bugs for Machine Learning Programs. In *24th Asia-Pacific Software Engineering Conference, APSEC 2017*. IEEE Computer Society.

[30] Lin Tan, Chen Liu, Zhenmin Li, Xuanhui Wang, Yuanyuan Zhou, and ChengXiang Zhai. 2014. Bug characteristics in open source software. *Em. Sof. Eng.* 19, 6 (2014).

[31] Ferdian Thung, Shaowei Wang, David Lo, and Lingxiao Jiang. 2012. An Empirical Study of Bugs in Machine Learning Systems. In *23rd IEEE International Symposium on Software Reliability Engineering, ISSRE 2012*. IEEE Computer Society.

[32] Yuan Tian, David Lo, Xin Xia, and Chengnian Sun. 2015. Automated prediction of bug report priority using multi-factor analysis. *Empir. Softw. Eng.* 20, 5 (2015).

[33] Saeid Tizpaz-Niari, Pavol Cerný, and Ashutosh Trivedi. 2020. Detecting and understanding real-world differential performance bugs in machine learning libraries. In *29th International Symposium on Software Testing and Analysis*.

[34] Jamal Uddin, Rozaida Ghazali, Mustafa Mat Deris, Rashid Naseem, and Habib Shah. 2017. A survey on bug prioritization. *Artif. Intell. Rev.* 47, 2 (2017).

[35] Shasha Wen, Xu Liu, John Byrne, and Milind Chabbi. 2018. Watching for Software Inefficiencies with Witch. In *Proc. of the 23rd International Conference on Architectural Support for Programming Languages and Operating Systems*. 332–347.

[36] Xin Xia, David Lo, Ming Wen, Emad Shihab, and Bo Zhou. 2014. An empirical study of bug report field reassignment. In *2014 Software Evolution Week - IEEE Conference on Software Maintenance, Reengineering, and Reverse Engineering*.

[37] Shahed Zaman, Bram Adams, and Ahmed E. Hassan. 2011. Security versus performance bugs: a case study on Firefox. In *Proc. 8th International Working Conference on Mining Software Repositories, MSR 2011*. ACM.

[38] Jie Zhang, Xiaoyin Wang, Dan Hao, Bing Xie, Lu Zhang, and Hong Mei. 2015. A survey on bug-report analysis. *Sci. China Inf. Sci.* 58, 2 (2015).

[39] Ru Zhang, Wencong Xiao, Hongyu Zhang, Yu Liu, Haoxiang Lin, and Mao Yang. 2020. An empirical study on program failures of deep learning jobs. In *ICSE 2020: 42nd International Conference on Software Engineering*. ACM.

[40] Yuhao Zhang, Yifan Chen, Shing-Chi Cheung, Yingfei Xiong, and Lu Zhang. 2018. An empirical study on TensorFlow program bugs. In *Proc. 27th ACM SIGSOFT International Symposium on Software Testing and Analysis, ISSTA 2018*. ACM.

[41] Yutong Zhao, Lu Xiao, Xiao Wang, Bihuan Chen, and Yang Liu. 2019. Localized or architectural: an empirical study of performance issues dichotomy. In *Proc. 41st International Conference on Software Engineering*.

[42] Yutong Zhao, Lu Xiao, Xiao Wang, Lei Sun, Bihuan Chen, Yang Liu, and Andre B. Bondi. 2020. How Are Performance Issues Caused and Resolved?-An Empirical Study from a Design Perspective. In *ICPE 2020: ACM/SPEC International Conference on Performance Engineering*. ACM.

[43] Yu Zhao, Feng Zhang, Emad Shihab, Ying Zou, and Ahmed E. Hassan. 2016. How Are Discussions Associated with Bug Reworking?: An Empirical Study on Open Source Projects. In *Proc. 10th ACM/IEEE International Symposium on Empirical Software Engineering and Measurement, ESEM 2016*. ACM.

[44] Thomas Zimmermann, Rahul Premraj, Nicolas Bettenburg, Sascha Just, Adrian Schröter, and Cathrin Weiss. 2010. What Makes a Good Bug Report? *IEEE Trans. Software Eng.* 36, 5 (2010).